AUTH-04 / STUDY CHAPTER 01

Understand and defend your AUTH-04 work

Younity frontend / Oussama / Jury preparation / 26 October 2026

A code-based learning guide

This guide explains what your registration frontend does, how its code works, why the design choices matter, and how to discuss its limits confidently. It is written for studying before a jury presentation, not as a claim that the complete authentication system is finished.

> **The central distinction:** a functioning registration interface is not the same as a working account-creation API. The current frontend has validated forms and submission boundaries, but real registration, uploads, role authorization and end-to-end backend integration are unfinished.

What this document is based on

The local branch **feature/auth-04-registration-ui**, committed baseline **ff77afc**, plus the uncommitted registration payload changes visible during this review. Code examples are taken from that working copy unless marked conceptual. The reference date is 26 October 2026.

Ownership

AUTH-04: registration and pending UI, form behavior, payload preparation, and ownership of shared API transport. **AUTH-05:** login/auth state, route guards, logout and auth tests. Their code has been integrated into your branch; do not present all merged code as work you individually authored.

How to study

First read chapters 2-5 for the big picture. Then trace a student submission in chapters 6-12 and an organizer submission in chapters 13-15. Finish with testing, integration gaps and jury questions. Explain each example aloud before memorizing syntax.

No source files were changed to write this guide. It describes the current state, including limitations and provisional mappings.

Reading map

Each chapter begins on a new page. Chapter 1 is the cover; chapter 2 begins on PDF page 3 because this reading map occupies page 2.

02 Your project in plain language / page 3

03 The implementation status / page 4

04 The architecture and file map / page 5

05 React, TypeScript and the browser / page 6

06 Draft types and typed field errors / page 7

07 Controlled inputs and data flow / page 8

08 Component reuse and composition / page 9

09 Validation: rules before submission / page 10

10 Password checks and visibility / page 11

11 Student flow: trace an actual submit / page 12

12 Async safety: state, refs and cancellation / page 13

13 Student preferences and retention / page 14

14 Organizer flow and state transitions / page 15

15 Document selection is not upload / page 16

16 Payload mapping: prepare, do not guess / page 17

17 Services, mocks and environment gates / page 18

18 Routing and AUTH-05 integration / page 19

19 Session state and the token mismatch / page 20

20 Accessibility and responsive design / page 21

21 Testing: what each layer proves / page 22

22 Current verification and source snapshot / page 23

23 Your finishing plan and team handoffs / page 24

24 A five-minute jury presentation / page 25

25 Jury questions with defensible answers / page 26

26 Glossary and code-reading checklist / page 27

27 Source index and revision notes / page 28

AUTH-04 / STUDY CHAPTER 02

Your project in plain language

Begin with the user problem, then explain your technical contribution.

Younity connects students with campus opportunities and allows organizers to submit their organization for verification. Your frontend contribution covers the entry point into those experiences: choosing an account type, entering information, receiving useful validation feedback, and seeing registration/application states.

Frontend, backend and database

• **Frontend:** React code running in the browser. It renders fields, responds to typing, validates input for usability and communicates with services.

• **Backend:** server code that must validate requests again, enforce permissions and create accounts. Your screen cannot establish that an account exists without a successful server response.

• **Database:** persistent storage owned through the backend. React state is temporary memory, not a database record.

What users can already do

Users can navigate between registration and login, enter and edit account information, see password checks, select a university or a custom institution, explore optional student preferences, enter organization information, and select a local proof document. Invalid inputs produce feedback rather than silently proceeding.

What users cannot yet rely on

Student account creation is unavailable. Organizer submission is simulated in development and blocked in production. A selected document is not uploaded. Preference choices are not saved. Example application status screens are not proof of a real review decision.

Say to the jury: "I implemented the registration user experience and its validation, component structure and submission lifecycle. I also integrated it with the shared frontend auth foundation. The remaining milestone is real API integration and verified end-to-end behavior."

AUTH-04 / STUDY CHAPTER 03

The implementation status

Use precise words: implemented, simulated, provisional and unfinished.

Area	Current evidence
Registration UI	Student and organizer forms, shared fields, validation and responsive styling are implemented.
Student submission	Validates and maps input, then calls a service that deliberately rejects as unavailable. No registration HTTP request is made.
Organizer submission	Validates input; development mode waits 1.5 seconds and returns a local pending state. Production rejects as unavailable.
Preferences	Optional selections and local retention work. Save/Skip are not connected to a backend completion flow.
Document handling	Selection, replacement, drag/drop and removal are local browser interactions. There is no real upload.
Router and provider	One BrowserRouter and one AuthProvider are mounted. /organizer/pending is guarded for sign-in.
Auth integration	Cookie-based client adapters exist. Role/status rules and the backend token strategy still need alignment.
Payload changes	payload.ts and payload.test.ts are new, untracked files; four existing registration files have local modifications.

Why a mock is useful

A mock lets you exercise loading, disabled controls, pending screens and cancellation before the server is ready. It is legitimate for development when visibly identified. It cannot prove persistence, permission enforcement, real uploads or correct server contracts.

Avoid a misleading demonstration

Say "the development simulation moved to pending" rather than "the organization was registered." Likewise, say "the file was selected" rather than "the document was uploaded." The latest production pending route displays unavailable status instead of confirming a submission.

AUTH-04 / STUDY CHAPTER 04

The architecture and file map

Separate page orchestration, reusable UI, rules, mappings and service boundaries.

```
main.tsx
  BrowserRouter
    AuthProvider                    AUTH-05 integration
      App.tsx                       one route table
        StudentRegisterPage         owns student drafts + submission
          AccountFields / Select    reusable controlled UI
          StudentPreferencesForm
          validateStudentDraft      pure validation
          mapStudentPayload         provisional DTO mapping
          registerStudent           service boundary, API unfinished
        OrganizerRegisterPage       owns org draft + File + status
          OrganizationFields
          mapOrganizerPayload
          registerOrganizer         development mock / production block
```

Why these responsibilities are separated

The page decides when to validate or submit. A field component renders a value and reports edits. A validation function returns errors without rendering anything. A mapper translates UI state into an API-shaped object. A service owns the operation boundary. This separation lets you change a backend payload without redesigning every input.

Location	Role
pages/	Composes a full screen and coordinates user actions.
components/	Reusable fields, layouts, selectors, feedback and application-state screens.
registration/	Draft types, validation, password rules, preference options, payload mapping and submission services.
auth/	Merged AUTH-05 session state, guard, auth HTTP adapters and tests.
test/setup.ts	Shared DOM matchers and automatic cleanup for tests.

Jury concept: this is separation of concerns, not a strict textbook MVC implementation. React combines view descriptions and event logic in components; pure domain helpers keep the most reusable rules outside JSX.

AUTH-04 / STUDY CHAPTER 05

React, TypeScript and the browser

Understand the tools before explaining their syntax.

React and JSX

A React function component returns a description of the interface. JSX resembles HTML but is compiled into JavaScript. When state changes, React calls the component again and reconciles the new output with the existing DOM. A re-render does not mean the browser reloads the whole page.

TypeScript

TypeScript checks the shapes and allowed uses of values during development/build. It can catch a boolean passed into an email setter, but it cannot guarantee that untrusted API JSON matches a type. Runtime checks and backend validation are still needed. Type information is erased from the browser JavaScript.

Vite and the build

Vite runs the development server and produces deployable assets. The build script runs **tsc -b** followed by **vite build**: type checking first, bundling second. The strict TypeScript configuration also rejects unused local variables and parameters.

Code source: apps/web/tsconfig.app.json:20

```
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
```

A useful mental model

Props are inputs to a component. State is data React preserves for that component across renders. Event handlers respond to user actions. Effects synchronize with something outside rendering, such as browser focus or cleanup. The DOM is the browser structure React updates.

What React does not do automatically

React does not create database users, secure an API endpoint, upload a file merely because it is in state, or persist drafts after refresh. Those behaviors require explicit application and backend code.

AUTH-04 / STUDY CHAPTER 06

Draft types and typed field errors

A draft models editable browser data, not a confirmed account.

Code source: apps/web/src/registration/model.ts:3

```
export type RegistrationDraft = {
  firstName: string
  customUniversity?: string
  university?: string
  lastName?: string
  email: string
  password: string
  confirmPassword: string
  acceptedPolicies: boolean
}
export type FieldErrors = Partial<Record<keyof RegistrationDraft, string>>
export type AccountType = 'student' | 'organizer'
export const emptyDraft: RegistrationDraft = {
```

The draft includes confirmPassword and acceptedPolicies because the UI needs them. That does not mean the backend accepts those keys. The student type extends the shared draft so lastName and university become required string properties at the type level.

Code source: apps/web/src/registration/model.ts:13

```
export type FieldErrors = Partial<Record<keyof RegistrationDraft, string>>
export type AccountType = 'student' | 'organizer'
```

Unpack the TypeScript expression

keyof RegistrationDraft is a union of draft property names. **Record<K, string>** associates those keys with strings. **Partial** makes every property optional. The result is an error object that contains only the fields currently failing validation.

```
// Conceptual example of a validation result
const errors: FieldErrors = {
  email: 'Enter a valid email address.',
  acceptedPolicies: 'Accept the policies to continue.',
}
```

Draft versus payload versus response

Draft: raw editable input, including confirmation and temporary UI details. **Payload:** an explicitly selected set of values sent to an endpoint. **Response:** data returned by the backend, such as a new user ID. Keeping these distinct prevents accidental leakage of UI-only fields or false assumptions about persistence.

> **Say:** "I use typed drafts to make field updates predictable, and typed error maps to connect validation results to the correct inputs."

AUTH-04 / STUDY CHAPTER 07

Controlled inputs and data flow

The page owns the value; the child reports edits.

Code source: apps/web/src/components/RegistrationForm.tsx:136

```
            value={draft[field] ?? ''}
            onChange={(event) => onChange(field, event.target.value)}
            aria-invalid={Boolean(errors[field])}
            aria-describedby={
              [
                errors[field] ? `${field}-error` : '',
                field === 'password' ? 'password-guidance-note' : '',
              ]
```

This input is controlled because its value comes from React state. Typing triggers onChange; the callback updates the parent draft; React re-renders with the new value. The nullish fallback keeps optional fields as strings in the input instead of switching between uncontrolled and controlled behavior.

Code source: apps/web/src/pages/StudentRegisterPage.tsx:52

```
  function change<K extends keyof RegistrationDraft>(field: K, value: RegistrationDraft[K]) {
    setMessage('')
    setDraft((current) => ({ ...current, [field]: value }))
    setErrors((current) => ({ ...current, [field]: undefined }))
  }
  function move(next: 1 | 2) {
```

Read the update carefully

The generic parameter K must be a valid draft key. RegistrationDraft[K] gives the matching value type. The functional setter receives the latest state, spreads its properties into a new object, and replaces only the edited property using the computed key [field].

A new object is created instead of mutating current[field]. Predictable immutable updates make React state transitions easier to reason about. The code also clears the edited field error and previous submission message, so stale feedback does not remain after a correction.

Why lift state into the page?

Account fields and preferences are child components that may appear or disappear when the step changes. Their durable draft values live in StudentRegisterPage, so switching between steps does not destroy them. Local state inside an unmounted child would be lost.

This is one-way data flow: parent state travels down as props; events travel back through callbacks. It is not automatic two-way binding, even though typing makes it feel synchronized.

AUTH-04 / STUDY CHAPTER 08

Component reuse and composition

Reuse behavior while allowing each screen to provide its own content.

RegistrationLayout

The shared layout handles the page shell, width variant, header, card, title and introduction. It receives children for the page-specific content. Optional beforeCard and headerContent props allow the organizer flow to supply a review notice and a different header without copying the whole layout.

Code source: apps/web/src/components/RegistrationForm.tsx:10

```
export function RegistrationLayout({
  title,
  introduction,
  children,
  beforeCard,
  headerContent,
  variant = 'selection',
}: {
  title: string
  introduction: string
  children: ReactNode
```

AccountFields

AccountFields renders shared labels, input types, autocomplete values, error messages and password controls. Student registration requests the last-name row. Organizer registration can omit the first-name field and supply organization details through its own surrounding components.

Composition means building a screen by combining smaller components. Props such as afterNames accept ReactNode content, allowing a university selector to be inserted after student names without adding university-specific logic to every account field.

RegistrationSelect

The selector receives options, current value, error text and an onChange callback. Its native select preserves standard keyboard interaction. A browser-specific showPicker branch handles mouse opening when available; fallback behavior remains native when unavailable. Tests check both paths.

Tradeoff

Reuse prevents inconsistent validation display and duplicated styling. Too many boolean props can make a component hard to understand, so new unrelated forms should not be forced into AccountFields just to avoid a small amount of markup.

> **Say:** "I reused the common field behavior, while the page components retain ownership of their own workflows. This keeps the student and organizer experiences consistent without making them identical."

AUTH-04 / STUDY CHAPTER 09

Validation: rules before submission

Frontend validation improves the experience; the server remains authoritative.

Code source: apps/web/src/registration/model.ts:32

```
export function validateDraft(
  draft: RegistrationDraft,
  includePolicies = true,
  includeFirstName = true,
): FieldErrors {
  const errors: FieldErrors = {}
  if (includeFirstName && !draft.firstName.trim()) errors.firstName = 'Enter your first name.'
  if (!draft.email.trim()) errors.email = 'Enter your email address.'
  else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(draft.email.trim()))
```

A pure validation function receives a draft and returns an error object. It does not submit a request or directly change component state. This makes the rules easy to test with ordinary values and reusable across rendering contexts.

Rules currently enforced

• Names and email have required/length checks; email has a basic format check. Student last name and university are required, with custom institution text required for Other.

• Password must meet all four current frontend checks; confirmation must match. Policy acceptance is required when included in validation.

• Organization name, representative, position, phone and category are validated. Other requires a custom category. Phone uses a permissive character-pattern check, not full international number validation.

How errors reach the interface

The page saves the error map, checks whether it contains any keys, focuses an invalid field and returns before creating a request. Each field reads its own error entry and renders an associated message. The form uses noValidate to let custom validation handle the submit path instead of the browser showing its default tooltip.

Limits to explain

The email regular expression does not verify mailbox ownership. Phone formatting does not prove a phone exists. Frontend policy acceptance is not a backend consent record. A client can bypass browser checks, so the server must validate again and return actionable errors.

> **Contract issue:** frontend password rules and backend DTO rules must agree. A successful local validation result is not evidence that the backend will accept the payload.

AUTH-04 / STUDY CHAPTER 10

Password checks and visibility

Derived feedback avoids duplicated, inconsistent state.

Code source: apps/web/src/registration/passwordStrength.ts:1

```
export function passwordChecks(password: string) {
  return [
    { label: '8+ characters', met: Array.from(password).length >= 8 },
    { label: 'At least 1 number', met: /[0-9]/.test(password) },
    { label: 'Special symbol', met: /[^\p{L}\p{N}\s]/u.test(password) },
    { label: '1 uppercase letter', met: /\p{Lu}/u.test(password) },
  ]
}
```

Array.from counts Unicode code points rather than raw UTF-16 units. It still does not count every user-perceived grapheme as one character. The number check uses ASCII digits; Unicode property escapes identify letters, numbers and uppercase letters in the other expressions.

Code source: apps/web/src/registration/passwordStrength.ts:10

```
export function meetsMinimumPasswordStrength(password: string) {
  return passwordChecks(password).every((check) => check.met)
}
```

One rule source, two uses

The feedback component displays each check and counts passing results. The validator uses every to enforce the same four checks. The current implementation therefore requires all four checks; older project notes describing them as advisory are historical and do not match this working copy.

Why score is not stored separately

The score can be computed from the current password during rendering. Storing both password and score in state would create two values that must always be synchronized. Deriving the score avoids stale strength bars after an edit.

Show/hide behavior

The visibility button changes the input type between password and text. It does not change the password string or submit the form because its type is button. aria-pressed communicates toggle state. Confirmation remains masked.

Security boundary

These checks are a UI policy, not proof of password entropy or a guarantee of security. The frontend does not hash passwords for database storage. Secure password hashing, validation, transport security and rate limits belong to backend/deployment responsibilities.

AUTH-04 / STUDY CHAPTER 11

Student flow: trace an actual submit

Follow the event from the button to the service boundary.

Code source: apps/web/src/pages/StudentRegisterPage.tsx:60

```
async function handleSubmit(event: FormEvent<HTMLFormElement>): Promise<void> {
  event.preventDefault()
  if (submissionRequest.current) return
  const nextErrors = validateStudentDraft(draft)
  setErrors(nextErrors)
  if (Object.keys(nextErrors).length) {
    focusFirstError(nextErrors)
    return
  }
  const controller = new AbortController()
  submissionRequest.current = controller
  setIsSubmitting(true)
  setMessage('')
```

1. preventDefault stops a full browser form navigation. **2.** submissionRequest.current blocks another request immediately. **3.** validateStudentDraft returns errors. **4.** the page displays errors, focuses the first one and returns if invalid. Only valid input reaches the asynchronous path.

Code source: apps/web/src/pages/StudentRegisterPage.tsx:69

```
  const controller = new AbortController()
  submissionRequest.current = controller
  setIsSubmitting(true)
  setMessage('')
  try {
    const payload = mapStudentPayload(draft, preferences)
    await registerStudent(payload, controller.signal)
    // TODO: Navigate to the approved completion route after confirmed API success.
  } catch {
    if (!controller.signal.aborted) setMessage(REGISTRATION_UNAVAILABLE)
  } finally {
```

5. A controller represents this request. **6.** isSubmitting disables the fieldset and communicates progress. **7.** mapStudentPayload selects API-shaped values. **8.** await pauses this handler until the Promise settles, while the browser remains able to render and process other events.

What happens today

registerStudent checks cancellation and then throws the explicit unavailable error. The catch block shows an unavailable message. No account is created and no HTTP registration request is issued. The success destination is still a TODO even after the real endpoint is connected.

Step 2 is a different action

Continue to preferences validates account information and changes the local step. It does not create an account. Preferences Save and Skip also remain local preview behavior. Decide with the team whether preferences are part of the registration request or a later authenticated onboarding step.

> **Say:** "The handler has the correct control flow for real integration, but the service intentionally blocks account creation until the API contract and completion behavior are approved."

AUTH-04 / STUDY CHAPTER 12

Async safety: state, refs and cancellation

These mechanisms solve different problems; they are not interchangeable.

useState versus useRef

isSubmitting is state because the interface must re-render when it changes. submissionRequest is a ref because it is a mutable handle the handler must read immediately without waiting for a render. React state setters schedule updates; relying only on a disabled button can leave a short window for repeated submission.

```
// Current pattern, simplified for explanation
if (submissionRequest.current) return
const controller = new AbortController()
submissionRequest.current = controller
setIsSubmitting(true)
```

AbortController

The controller owns a signal. Cleanup calls abort when the page unmounts. A future fetch must receive this signal to participate in cancellation. The development mock explicitly listens for abort and clears its timer. A plain Promise is not automatically cancellable.

Code source: apps/web/src/pages/StudentRegisterPage.tsx:45

```
  useEffect(() => {
    return () => submissionRequest.current?.abort()
  }, [])
```

try, catch and finally

try contains the operation, catch handles failure, and finally releases the active-request handle and loading state when appropriate. The signal checks prevent an obsolete response from updating a page that has already been left.

Important limits

Aborting a browser request does not roll back a server transaction that has already committed. Browser duplicate-submit prevention also does not guarantee server-side uniqueness. The backend still needs appropriate constraints or idempotency behavior. In the current code, broad catch blocks show a generic unavailable message; real integration should preserve field, conflict, network and server errors separately.

Complexity

Updating a draft copies a small object; validating fields is linear in the number of inspected fields and input characters. These are small forms, so correctness and understandable transitions matter more than micro-optimizing their rendering.

AUTH-04 / STUDY CHAPTER 13

Student preferences and retention

Optional choices are local state until an API saves them.

Code source: apps/web/src/registration/studentPreferences.ts:37

```
export type StudentPreferences = {
  discipline: string
  academicLevel: string
  interests: string[]
  formats: string
}
```

Discipline and academic level hold one selected label each. Interests is an array so several interests can be selected. Formats currently holds one string, so the current UI model allows one selected format. Empty values represent optional unanswered questions.

Selection behavior

The component updates the whole preference object through onChange. Exclusive choices keep one value; toggling a selected option can clear it. Interests add/remove values in an array. The validator rejects unsupported values and duplicate interests, but it does not require the old minimum of two interests.

Code source: apps/web/src/registration/studentPreferences.ts:50

```
export function validatePreferences(preferences: StudentPreferences): PreferenceErrors {
  const errors: PreferenceErrors = {}
  if (
    preferences.discipline &&
    !disciplines.some((option) => option.label === preferences.discipline)
  )
    errors.discipline = 'Choose your primary field of study.'
  if (
    preferences.academicLevel &&
    !academicLevels.some((option) => option === preferences.academicLevel)
```

Why Back preserves selections

StudentRegisterPage owns both the account draft and preferences. Switching the rendered child changes the screen without destroying that parent. A page refresh or route unmount normally resets these in-memory drafts; the code does not establish persistent browser storage.

Labels versus API enums

A friendly label such as AI & Data Science is presentation data. An API enum such as AI_DATA is a contract value. The mapping must be explicit and agreed; uppercasing arbitrary text does not reliably create a valid enum.

Current mapping limitation

The provisional payload sends only the first selected interest and omits discipline, academic level and format. This is a data-loss tradeoff, not complete preference integration. The team must either support arrays on the backend or revise the UI/contract together. Do not silently discard additional user selections in a production flow.

AUTH-04 / STUDY CHAPTER 14

Organizer flow and state transitions

Local application screens are a presentation state machine.

Code source: apps/web/src/registration/applicationState.ts:1

```
export type ApplicationState = 'form' | 'pending' | 'approved' | 'rejected' | 'needs_edit'
```

This string union restricts the allowed local states. The form starts in form. A successful development simulation returns pending. Other screens demonstrate approved, rejected and needs_edit states. These strings express UI states, not trusted permission grants.

What the page owns

OrganizerRegisterPage owns account information, organization details, selected File, policy acceptance, field errors, loading state, submission errors and the current application state. Child components render those values and report edits.

Validation before simulation

validateForm combines account validation and organization validation. The organizer currently omits a separate first-name input, so validateDraft is called with includeFirstName=false. Email, password, confirmation and policy checks still apply. The current mapper derives account names from the representative name, which requires backend/product agreement.

Code source: apps/web/src/pages/OrganizerRegisterPage.tsx:78

```
      const payload = mapOrganizerPayload(account, organization, document)
      const nextState = await registerOrganizer(payload, controller.signal)
      if (!controller.signal.aborted) setState(nextState)
    } catch {
      if (!controller.signal.aborted) setSubmissionError(REGISTRATION_UNAVAILABLE)
    } finally {
      if (!controller.signal.aborted) {
```

Development and production

In development, registerOrganizer resolves pending after a simulated delay. The page changes its local state and renders the application-status component at the registration URL. It does not navigate through the guarded standalone pending route. In production the service rejects, so a real submission is not claimed.

Integration gap: displaying local pending under /register/organizer is distinct from loading a real signed-in application at /organizer/pending. Only the latter route is currently wrapped by the auth guard. Neither local state nor frontend visibility replaces backend authorization.

AUTH-04 / STUDY CHAPTER 15

Document selection is not upload

Understand File objects, browser events and the missing server operation.

What a File represents

The browser provides a File object when a user chooses a document or drops one into the page. It contains metadata and access to the selected bytes. Storing it in React state does not transmit those bytes to a server or save them permanently.

Code source: apps/web/src/pages/OrganizerRegisterPage.tsx:48

```
function selectDocument(file: File | undefined) {
  if (!file || submissionRequest.current) return
  setDocument(file)
  if (fileInput.current) fileInput.current.value = ''
}
function changePolicies(value: boolean) {
```

The hidden file input is referenced with useRef. Clearing its value allows choosing the same file again. Browse buttons open the picker; dropping a file reads dataTransfer.files[0]. The surrounding click behavior avoids double activation when the target is already a button or input.

What a real upload needs

The team must define an upload endpoint or signed-storage flow, allowed types and sizes, authorization, error behavior and a server-issued reference/URL. A typical multipart request uses FormData, but that is a future option rather than current implementation. Server validation is required even if the browser filters file types.

Current provisional proofUrl

The payload mapper substitutes https://mock-storage.com/proof.pdf when a File is selected. That is a hard-coded demonstration value, not a URL produced by uploading the selected bytes. It must be replaced before live integration. File names and local File objects must not be presented as server-confirmed evidence.

What the jury may ask

"Where is the document stored?" Answer: "Currently it is only selected in browser memory. Persistent upload is unfinished. The final registration payload must reference a real uploaded document according to the agreed backend contract."

AUTH-04 / STUDY CHAPTER 16

Payload mapping: prepare, do not guess

This chapter describes uncommitted work in the reviewed working copy.

Code source: apps/web/src/registration/payload.ts:41

```
export function mapStudentPayload(
  draft: StudentRegistrationDraft,
  preferences: StudentPreferences,
): StudentRegistrationPayload {
  const firstInterest = preferences.interests[0]
  const interests = Object.entries(interestCodes).find(([label]) => label === firstInterest)?.[1]
  return {
    firstName: draft.firstName.trim(),
    lastName: draft.lastName.trim(),
    email: draft.email.trim().toLowerCase(),
    password: draft.password,
    university: (draft.university.toLowerCase() === 'other'
      ? draft.customUniversity ?? ''
      : draft.university).trim(),
    // The DTO uses the plural key `interests` but accepts a single enum value.
    ...(interests ? { interests } : {}),
```

The mapper trims names, normalizes email casing, substitutes a custom institution for Other and maps a display interest into an enum. Password is preserved exactly; trimming it would change the user secret. Confirmation and policy checkbox are omitted because they are not part of this provisional DTO.

Why explicit mapping helps

Sending the whole draft with object spread can include UI-only keys and trigger strict DTO rejection. A mapper makes the boundary reviewable and testable. However, a TypeScript interface only proves consistency with the interface you wrote, not with the backend controller actually deployed.

Organizer issues to resolve

• Splitting representativeFullName at the first space invents first/last-name semantics and produces an empty last name for a single-word name.

• customType is conditionally included for Other even though the inspected backend strict DTO does not accept that property.

• proofUrl is a mock constant when a document exists; it is not an uploaded artifact.

• Organization category values and all name/password/phone limits must be checked against the approved backend version.

Student issues to resolve

Only the first interest is retained; other preference fields are omitted. Frontend and backend requirements for university and passwords differ in the inspected branch snapshots. These are coordination issues, not details to hide behind a successful frontend type check.

> **Say:** "The mapper is a provisional integration layer. Its tests document current transformations, but we still need contract agreement before treating it as a production DTO implementation."

AUTH-04 / STUDY CHAPTER 17

Services, mocks and environment gates

A service boundary keeps API operations separate from rendering.

Code source: apps/web/src/registration/service.ts:11

```
export async function registerStudent(
  payload: StudentRegistrationPayload,
  signal: AbortSignal,
): Promise<void> {
  signal.throwIfAborted()
  // Retain the typed input boundary until the approved DTO mapping is supplied.
  void payload
  // TODO: Connect real API. Map approved DTO fields explicitly; never send
  // confirmPassword or assume UI institution labels are backend identifiers.
  throw new Error(REGISTRATION_UNAVAILABLE)
}
```

Today the student service accepts a typed payload and an AbortSignal, checks cancellation, and rejects with REGISTRATION_UNAVAILABLE. This stable function signature lets the page exercise validation and error handling without pretending that a backend exists.

Code source: apps/web/src/registration/service.ts:42

```
export async function registerOrganizer(
  _payload: OrganizerRegistrationPayload,
  signal: AbortSignal,
): Promise<'pending'> {
  if (import.meta.env.DEV) return mockRegisterOrganizer(signal)
  // TODO: Connect real API with fetch(..., { signal }). Map the approved DTO
  // and upload contract; return pending only after confirmed server success.
  throw new Error(REGISTRATION_UNAVAILABLE)
}
```

How the mock works

mockRegisterOrganizer creates a Promise and schedules a timer. It rejects immediately if already aborted, adds an abort listener for cancellation, clears the timer on cancellation, and removes the listener when the timer resolves. Tests use fake timers so a 1.5-second simulation can be checked deterministically.

What import.meta.env.DEV means

Vite supplies a development/build flag. It gates preview behavior and the organizer mock in the current implementation. It is not a permission check, an approval from a server, or a substitute for access control.

Future shared transport

AUTH-04 owns the shared HTTP transport, but this branch does not yet have a completed common transport used by registration and auth. A shared transport should centralize the agreed base URL, authentication attachment, request serialization, cancellation and error interpretation. AUTH-05 should own how auth state reacts to authentication failures.

AUTH-04 / STUDY CHAPTER 18

Routing and AUTH-05 integration

Explain what was merged without claiming the other developer's ownership.

Code source: apps/web/src/main.tsx:10

```
<BrowserRouter>
  <AuthProvider>
    <App />
  </AuthProvider>
</BrowserRouter>
```

BrowserRouter synchronizes navigation with the browser URL. AuthProvider shares session information across the routed app. Keeping one instance of each avoids separate navigation contexts or inconsistent signed-in users in different parts of the interface.

Code source: apps/web/src/App.tsx:17

```
    <Route element={<RequireAuth />}>
      <Route path="/organizer/pending" element={<OrganizerPendingPage />} />
    </Route>
```

How the guard works

RequireAuth shows a loading message while checking the session. Anonymous users are redirected to /login and the requested pending path is saved in navigation state. Session errors block protected content and offer retry. Signed-in users render the nested route through Outlet, with a logout control.

Authentication versus authorization

Authentication answers "Who is this user?" **Authorization** answers "May this user perform this action?" Current session data contains ID and email only. Therefore the pending guard verifies sign-in but cannot enforce organizer approval rules. The backend must independently enforce authorization on data and actions.

Current routing gaps

StudentDashboardPage and OrganizerDashboardPage exist, but App.tsx does not register their URLs. Successful login returns to the saved pending route when allowed, otherwise /register. Role/status dashboard destinations remain AUTH-05 work coordinated with the shared route table.

> **Ownership language:** "I integrated my registration routes with AUTH-05's session foundation. We kept one router, one provider and one guarded standalone pending route."

AUTH-04 / STUDY CHAPTER 19

Session state and the token mismatch

The integration is structurally present, but its backend contract is unresolved.

Auth provider states

The provider distinguishes loading, authenticated, anonymous and error. It checks /api/auth/me at startup and exposes signIn, signOut and refreshSession. A request counter prevents an older response from overwriting a newer session transition. This matters when a slow startup request completes after logout.

Current client assumption

The auth adapter uses credentials: include, which permits cookie credentials under browser rules. After login it calls /api/auth/me to confirm the session. Logout calls POST /api/auth/logout. These requests do not attach a bearer access token.

Previously inspected AUTH-03 branch

The locally available backend snapshot returns an accessToken in login JSON and extracts JWTs from the Authorization bearer header. Its /auth/me response exposes only ID/email and it has no logout controller endpoint. Thus the current cookie client cannot be called fully integrated with that bearer backend. These are branch-snapshot findings, not a fresh live-server verification.

Decisions AUTH-05 and AUTH-03 must make

• Approve the token/cookie strategy, persistence and startup restoration behavior. Do not casually introduce localStorage or assume refresh tokens exist.

• Return the role and agreed approval/status fields needed for frontend routing. Reconcile naming differences between registration and JWT branches.

• Define logout semantics and expired-session handling for protected requests, including what a 401 or 403 means for state and navigation.

Frontend guards improve navigation and avoid showing inappropriate UI. They cannot make an unprotected backend endpoint secure. Conversely, hiding a page during a failed request is not proof that the server session was revoked.

AUTH-04 / STUDY CHAPTER 20

Accessibility and responsive design

These are concrete implementation behaviors, not just styling claims.

Accessible form structure

Labels use htmlFor to identify inputs. Error text has stable IDs. aria-invalid marks invalid fields, and aria-describedby links fields with help/error content. Fieldsets and legends group related input. Status regions announce asynchronous feedback without requiring a user to search the page.

Code source: apps/web/src/registration/focus.ts:4

```
export function focusFirstError(errors: FieldErrors) {
  const field = [...studentAccountFields, 'university' as const, 'customUniversity' as const,
'acceptedPolicies' as const].find(
    (key) => errors[key],
  )
  if (field) document.getElementById(field)?.focus()
}
```

Focus after errors and step changes

focusFirstError selects the first invalid field in a defined order. useLayoutEffect focuses the page heading after a step/status change. The heading has tabIndex=-1 so code can focus it without adding an unnecessary tab stop. Focus management should be verified with a keyboard, because a DOM test alone does not fully reproduce a screen reader.

Native controls and keyboard behavior

Native inputs/selects retain browser keyboard semantics. Password visibility is an actual button. File browsing uses a real file input. The selector fallback preserves normal operation where showPicker is not available. Decorative icons are hidden from assistive technology when appropriate.

Responsive styling

Shared CSS defines page/card styles, tokens and layout rules; the registration stylesheet and Tailwind utilities implement screen-specific layouts. Grid/flex and breakpoint rules stack rows and cards on narrower screens. Password and phone controls need enough room for text, toggles and errors.

Be accurate about verification

Source code shows accessibility intent. It does not prove complete accessibility conformance. Browser checks should include keyboard-only operation, focus visibility, zoom, narrow screens, long error text, and the production build. Earlier screenshot records are historical, not a new visual test of today's working copy.

AUTH-04 / STUDY CHAPTER 21

Testing: what each layer proves

A passing test has a specific scope; explain that scope to the jury.

Test layer	What it demonstrates
Pure unit tests	Password checks, validation results, payload transformations and optional-field behavior for supplied inputs.
Component tests	Typing, labels, visibility toggles, selections, focus, retained drafts and error rendering using Testing Library.
Service/mock tests	Production unavailable behavior, delayed mock success, timer cleanup and cancellation using Vitest fake timers.
Auth integration tests	Provider and guard behavior, login return flow, logout failures and stale session responses with injected services.
Real end-to-end tests	Still required: actual registration, database effects, real login/session, correct role/status destination and expired-session behavior.

Code source: apps/web/vitest.config.ts:6

```
test: {
  environment: 'jsdom',
  setupFiles: ['./src/test/setup.ts'],
  include: ['src/**/*.test.{ts,tsx}'],
},
```

jsdom supplies a DOM-like environment in Node; it is not a full browser. Shared setup imports jest-dom matchers and cleans up rendered components after each test. Testing Library queries accessible roles/labels so tests resemble user interaction rather than depending on internal state variables.

The three local commands

```
cd apps/web
npm test          # behavior under the configured test runner
npm run lint      # static rules and React hook checks
npm run build     # TypeScript checking + production bundling
```

A limitation in payload tests

A test that expects only the first interest proves that the current mapper behaves that way. It does not prove that discarding other interests is a correct product decision. Contract tests need independently agreed API expectations, not merely copied implementation assumptions.

AUTH-04 / STUDY CHAPTER 22

Current verification and source snapshot

Read this page as evidence, not a blanket claim that the product is complete.

Test Result

PASS: 10 test files, 57 tests. npm test ran against the current working copy while preparing this guide.

Lint Result

PASS: npm run lint completed with no reported errors.

Production Build

PASS: npm run build completed TypeScript checking and Vite asset bundling.

Scope Of Evidence

Local automated checks on 26 October 2026. No real backend registration/upload journey, live authentication compatibility test, or new browser accessibility audit was performed for this document.

Local change boundary

The payload mapping work is uncommitted in the reviewed snapshot: registration/payload.ts and payload.test.ts are new files, and StudentRegisterPage.tsx, OrganizerRegisterPage.tsx, registration/service.ts and service.test.ts are modified. The baseline commit is ff77afc. This guide does not commit, push or modify those source changes.

What a build establishes

A successful build establishes that TypeScript checks and asset bundling completed under this environment. It does not establish API availability, token compatibility, database correctness, browser accessibility or production deployment.

What must be demonstrated before completion

- Real student creation, duplicate-email and invalid-field handling.
- Real organizer creation and upload, followed by an authenticated pending view based on server data.
- Correct dashboard routing for student, pending organizer and approved organizer.
- Session refresh, rejected/expired credentials, unauthorized routes and verified logout behavior.
- CI, browser-console/server-error review and the assigned reviewers' approval.

AUTH-04 / STUDY CHAPTER 23

Your finishing plan and team handoffs

Sequence the work so UI choices and backend contracts agree.

Owner	Concrete next step
AUTH-04 + AUTH-02	Approve exact student/organizer payloads, validation, optional fields, enum values and server error shape. Resolve names, multi-interest selections and customType.
AUTH-04	Replace fake proofUrl with a server-issued upload result. Connect registration services through shared transport. Preserve field errors and define real loading/retry behavior.
AUTH-03 + AUTH-05	Resolve bearer versus cookie strategy, restoration/expiry/logout behavior and the /me role/status response.
AUTH-05	Implement role/status guards and dashboard routes. Invalidate shared state on expired sessions according to the approved contract.
AUTH-04 + AUTH-05	Agree post-registration navigation, login handoff, optional-preferences timing and one route definition per page.
Both + reviewers	Run real journeys, record defects, resolve review comments and provide CI/build/browser evidence before moving cards to Testing.

Remove development assumptions deliberately

Keep mock transitions explicit during development, but replace them with confirmed server results for the real flow. Resolve placeholder Terms/Privacy destinations and example application IDs/status text. A signed-in user must not see another person's dossier; identity and ownership must be enforced by backend data access.

How to report progress

Separate completed UI work, locally passing verification, provisional mappings and externally blocked contracts. Avoid a single completion percentage that mixes all four. This makes the remaining work assignable and keeps the review discussion factual.

AUTH-04 / STUDY CHAPTER 24

A five-minute jury presentation

Use this structure, then demonstrate only what the current code supports.

Minute 1: problem and scope

"My AUTH-04 task covers the student and organizer registration experience. I built the forms, validation, preference controls, local document selection and application-state UI. AUTH-05 owns session state and protected navigation; we integrated the two into one frontend structure."

Minute 2: architecture

"Pages coordinate the workflow. Reusable components render controlled inputs. Pure functions validate drafts. Payload mappers prepare explicit API objects, and services isolate submission. This lets me test the rules separately from the layout and change integration code without rewriting every field."

Minute 3: technical example

Show StudentRegisterPage.handleSubmit. Explain preventDefault, error mapping, focus, request lock, loading state, AbortController, await/catch/finally and the service boundary. Explain why useRef blocks repeated requests immediately while useState updates what the user sees.

Minute 4: demonstration

Show invalid student fields and focus, live password checks, university Other, retained preferences and the unavailable submission message. If showing organizer submission, announce development simulation before clicking. Explain that selected documents remain local and example status is not a confirmed approval.

Minute 5: evidence and next step

"Tests cover form behavior, pure validation, payload transformations, cancellation and auth integration. Real API journeys remain the next milestone. We must finalize DTOs, upload and bearer/cookie strategy, add role/status routing, then demonstrate both journeys against the integrated backend."

Presentation habit: explain one decision, show the relevant code, describe what a test proves, and state the limitation. This is more defensible than listing framework names or reading JSX line by line.

AUTH-04 / STUDY CHAPTER 25

Jury questions with defensible answers

Practice these in your own words, then point to the relevant file.

Why React? Why not manipulate every input with DOM calls?

React lets the UI be derived from state and composed from reusable components. I use DOM access narrowly for focus and file-picker interaction; I do not manually synchronize the whole form DOM after every edit.

Why are drafts in the parent?

Because account and preference children can unmount when switching steps. Keeping values in the page preserves them while that page stays mounted. It also gives submission one consistent source of truth.

Why validate again on the backend?

Browser validation is bypassable. Client validation improves feedback; backend validation protects storage and business rules. They must agree, but neither replaces the other.

What does a Promise do? What does await do?

A Promise represents a future result or failure. await pauses this async function until it settles; it does not block the entire browser. Loading state keeps the user informed while the operation is pending.

Does your application currently register users?

Not through these registration services. Student registration rejects as unavailable; organizer development mode simulates pending and production rejects. I can explain the UI and lifecycle without claiming database persistence.

Are the routes secure?

The pending route currently checks sign-in. Role/status authorization is incomplete and all protected API actions require server enforcement. A frontend guard alone is not a security boundary.

What would you improve first?

Finalize the backend contracts, remove provisional lossy/fake payload mappings, connect upload and registration, align authentication transport with AUTH-03, then verify the real role/status journeys with AUTH-05.

AUTH-04 / STUDY CHAPTER 26

Glossary and code-reading checklist

Use precise definitions instead of memorizing unexplained vocabulary.

Term	Meaning in this project
Component	A React function that describes a reusable piece of UI.
Props / state	Inputs supplied by a parent / data React preserves for a mounted component.
Hook	A React API such as useState, useRef or useEffect, called under React's hook rules.
Controlled input	An input whose current value is supplied from React state.
Pure function	A function that derives a result from inputs without changing external state.
DTO / payload	An API data contract / the concrete request object prepared for that contract.
Enum / union	A constrained set of accepted values, represented in TypeScript or backend schema.
Promise / async	A future completion or failure / syntax for functions returning promises.
Ref	A persistent mutable holder that does not itself trigger a render.
AbortSignal	A notification channel for requesting cancellation of participating async work.
Mock	A controlled replacement for a real dependency, useful for development or tests.
Authentication / authorization	Establishing identity / checking allowed actions and resources.
Router / provider	URL-to-screen navigation context / a shared context value for descendants.
Unit / integration / E2E	Small isolated rule / interacting parts / a complete real user journey.

For every file you present, answer these five questions

What data enters? Who owns that data? What event changes it? What side effect can happen? What evidence proves the behavior, and what remains unverified? If you can answer those five questions, you understand the code beyond its syntax.

AUTH-04 / STUDY CHAPTER 27

Source index and revision notes

Local source references for independent review.

Paths below are relative to the repository root. Code excerpts elsewhere include the starting line in the reviewed working copy. Line numbers may shift after edits. The code, rather than older README descriptions, is the basis for current behavior in this guide.

apps/web/src/App.tsx
apps/web/src/components/RegistrationForm.tsx
apps/web/src/main.tsx
apps/web/src/pages/OrganizerRegisterPage.tsx
apps/web/src/pages/StudentRegisterPage.tsx
apps/web/src/registration/applicationState.ts
apps/web/src/registration/focus.ts
apps/web/src/registration/model.ts
apps/web/src/registration/passwordStrength.ts
apps/web/src/registration/payload.ts
apps/web/src/registration/service.ts
apps/web/src/registration/studentPreferences.ts
apps/web/tsconfig.app.json
apps/web/vitest.config.ts
apps/web/src/registration/payload.test.ts
apps/web/src/registration/service.test.ts
apps/web/src/auth/AuthProvider.tsx
apps/web/src/auth/RequireAuth.tsx
apps/web/src/auth/service.ts
apps/web/src/auth/auth.test.tsx
apps/web/src/components/StudentPreferencesForm.tsx
apps/web/src/components/OrganizerApplicationStatus.tsx
apps/web/src/pages/OrganizerPendingPage.tsx
apps/web/src/index.css
apps/web/src/registration/stitch.css
apps/web/src/test/setup.ts

External task reference supplied by the user

AUTH-05 - Create login, protected routes and end-to-end integration. Trello card: https://trello.com/c/pdOerbgb. Its pasted export was reviewed earlier in the conversation; this document does not claim a fresh live Trello inspection. AUTH-04 scope follows the agreed ownership and local source, not a newly retrieved card.

What changed since older records

Current password checks are enforced, preferences are optional, the organizer flow is a single form with state screens rather than the historical wizard, production pending is unavailable, and payload mapping is now present as uncommitted work. Historical counts and demonstrations must not be reused as current evidence.